SchedularOS — API Reference

Base URL: `http://localhost:8000/api/v1`

Auth: `Authorization: Bearer <access>` (JWT, obtained via `/auth/token/`)

Multi-tenancy: `request.org` is resolved from `UserProfile.org` — every query is org-scoped.

List endpoints return DRF pagination: `{count, next, previous, results}` unless noted.

---

1. Auth (`core/auth_urls.py`)

Method	Path	Body → Response	Purpose
POST	`/auth/token/`	`{username, password}` → `{access, refresh}`	JWT obtain pair (simplejwt)
POST	`/auth/token/refresh/`	`{refresh}` → `{access}`	Refresh access token
POST	`/auth/register/`	`{username, password, email, org_name}` → `{message, user_id, org_id}`	Create user + org + admin profile
GET	`/auth/me/`	— → `{id, user, org: {id,name,slug,settings}, role}`	Current profile (deps: `UserProfile`, `OrganizationSerializer`)

---

2. Core (`core/urls.py` → `core/views.py`)

`OrganizationViewSet` — read-only

Method	Path	Purpose
GET	`/organizations/`	List orgs the user belongs to

`BrandViewSet` — CRUD

Method	Path	Schema	Purpose
GET/POST	`/brands/`	`{name, voice, rules, colors, logo_url}` → same + `id`	Brand voice/rules injected into agent prompts
GET/PUT/DELETE	`/brands/{id}/`	same	Manage a brand

`UserProfileViewSet` — read-only

| GET | `/profiles/` | → `{id, user, org, role}` | Team members of the org |

`LeadViewSet` — CRUD (`?stage=` filter)

| GET/POST | `/leads/` | `{name, email?, phone?, stage, score?, tags?, has_whatsapp_consent?, has_email_consent?}` → same + `id, opted_out_at, last_contacted_at` | Pipeline data source |

- Functionality: leads are what agents act on (whatsapp/email outreach, scoring, tagging via `lead.*` proposals).

`ResearchViewSet` — `POST /research/perform/`

- Body: `{research_type: market|competitors|audience, industry?, company_name?, product?, company_context?, async_mode?}`
- Returns: `{topic, findings, sources[], confidence, metadata}` or `{task_id, status:"processing"}` when `async_mode`
- Dependencies: sync → `packages/agents/research_agent.ResearchAgent` (`research_market/_competitors/_audience`); async → `core.tasks.async_research_task` (Celery).

`WorkflowViewSet` — `POST /langgraph/run/` (standalone LangGraph pipeline, NOT the agentic engine)

- Body: `{workflow_type: orchestrator|marketing, goal, industry?, company_name?, product?, company_context?, async_mode?}`
- Returns: `{goal, current_step, research_results[], proposals[], errors[], context{}}`
- Dependencies: `packages/orchestration/graph.py` `OrchestratorGraph` / `MarketingWorkflowGraph`; async → `async_workflow_task`.

`KnowledgeViewSet`

| POST | `/knowledge/store/` | `{title, content, source?, metadata?, async_mode?}` → `{id, message}` / `{task_id}` | Dep: `memory/rag.get_rag_service().store_document` (pgvector, OpenAI embeddings w/ mock fallback) |
| POST | `/knowledge/search/` | `{query, top_k?}` → `{results: [{id?, title, content, score}]}` | Dep: `rag_service.search_documents` → `memory/retrieval.semantic_search` |

Webhooks (no auth; `csrf_exempt`)

| POST | `/webhooks/whatsapp/` | Meta Cloud-API payload `{entry:[{changes:[{value:{messages:[...]}}]}]}` → `{status, processed:[{from,status}]}` | Dep: `Lead` lookup by phone, `Conversation` / `Message` create, `orchestration.engine.create_workflow` (conversation agent), `external_id` dedup |
| GET/POST | `/webhooks/whatsapp/verify/` | Meta `hub.challenge` verification |

---

3. Orchestration (`orchestration/urls.py`)

`WorkflowRunViewSet`

Method	Path	Body → Response	Dependencies
GET	`/workflows/`	→ paginated `WorkflowRun[]` `{id, goal, status, lead, brand, context, steps[], created_at, updated_at}`	—

Method	Path	Body → Response	Dependencies
GET	`/workflows/{id}/`	→ single `WorkflowRun` with nested `steps`	–
POST	`/workflows/launch/`	`{goal}` → `{workflow_id, status:"planning"}`	`orchestration.tasks.create_workflow_from_goal` (Celery) → `OrchestratorAgent.plan` (LLM) → `engine.create_workflow`
POST	`/workflows/{id}/cancel/`	→ run + pending steps cancelled	`RunStep.mark_cancelled`
POST	`/workflows/{id}/pause/`	→ status `paused`	–
POST	`/workflows/{id}/resume/`	→ status `running` + re-dispatch due steps	`run_step.apply_async`

`RunStepViewSet` — read-only (`?run=` filter)

- GET `/steps/` → `{id, run, agent, state, run_at, started_at, completed_at, attempt, input_context, output, error_message}` ordered by `run_at`.
- Dependencies: produced/consumed by `run_step` Celery task → `build_run_context` → `BaseAgent.run` (LangGraph) → `submit_proposal` → `advance_run`.

---

4. Proposals (`proposals/urls.py`)

`ProposalViewSet` — read-only + decide (`?status=` filter)

GET	`/proposals/`	→ paginated `{id, step, action_type, payload{}, rationale, evidence{}, risk_tier 0-3, status, expires_at, executed_at, error_message, approval{decided_by,decision,note,edited_payload,created_at}, created_at, updated_at}`
GET	`/proposals/{id}/`	same
POST	`/proposals/{id}/decide/`	`{decision: approved|rejected|edited, note?, edited_payload?}` → `{proposal_id, status, decision}`

- `decide` dependencies: `Approval` row + `AuditEvent` + `update_trust` (edited counts as approval) + `execute_proposal.apply_async` (approved/edited) + `engine.resolve_step_after_decisions` (workflow advancement; rejection requires `note`).

---

5. Agents (`agents_api/urls.py`)

| GET | `/agents/` | → `{agents: [{name, model_tier, queue, default_risk_tier}]}` | Dep: `agents.registry.list_agents/get_agent` |
| POST | `/agents/run/` | `{agent, goal, lead_id?, brand_id?}` → `{workflow_id}` (202) | Dep: `Lead` / `Brand` lookup, `engine.create_workflow` (single-step run)|

---

6. Conversations (`conversations/urls.py`)

| GET | `/conversations/` | → `[{id, lead:{id,name,stage}, channel, status, last_message{body,direction,created_at}, created_at}]` | Dep: `Conversation` + `related_name=messages` |
| GET | `/conversations/{id}/messages/` | → `[{id, direction, body, created_at}]` | Dep: `conv.messages.order_by(created_at)` |

---

7. Analytics (`analytics/urls.py` → `AnalyticsViewSet`)

All GET, org-scoped, real computed data only (`available:false` instead of fabricated values where no source exists).

Path	Returns	Depends on
/analytics/kpis/	{total_leads, won_leads, conversion_rate, leads_by_stage{}, total_spend_paise, active/completed/failed_workflows, pending/executed/rejected_proposals, conversations, messages_inbound/outbound, data_quality{}}	Lead, WorkflowRun, Proposal, Conversation, Message
/analytics/lead_trend/?days=N	{series:[{date,count}]}	Lead.created_at
/analytics/funnel/	{stages:[{stage,count}]} ordered new→lost	Lead.stage
/analytics/lead_sources/	{sources:[{name,count}], note} — grouped by lead tags	Lead.tags
/analytics/lead_score_distribution/	{buckets:[{range,count}]} 0-100 in 5 buckets	Lead.score
/analytics/agent_productivity/	{agents:[{agent, scheduled, running, done, awaiting_human, failed, cancelled, agent_runs, proposals}]}	RunStep, AgentRun
/analytics/workflow_stats/	{by_status[], step_states{}, avg_completion_seconds}	WorkflowRun, RunStep
/analytics/propos	{by_status[], by_risk_tier[], top_actions[], decisions{},	Proposal, Approval

Path	Returns	Depends on
`al_stats/`	`avg_decision_seconds}`	
`/analytics/spend_over_time/?days=N`	`{series:[{date,spend_paise}]}` — executed proposals only	`Proposal.executed_at`, `payload.spend_paise`
`/analytics/activity/?limit=N`	`{events:[{id,event_type,proposal_id,metadata,created_at}]}`	`AuditEvent`
`/analytics/message_trend/?days=N`	`{inbound:[{date,count}], outbound:[{date,count}]}`	`Message`
`/analytics/trust/`	`{trust:[{agent,action_type,trust_level,approvals,rejections,total_decisions}]}`	`AgentTrustState`
`/analytics/integrations/`	`{integrations:[{name,category,configured}]}` — booleans only, no secrets	`settings.*_CONFIG`
`/analytics/ad_campaigns/`	`{campaigns:[{id,name,status,objective,budget,spend,impressions,clicks,conversions}], available}` or `{available:false, reason}`	`connectors.meta_connector.get_meta_connector().get_campaigns()`

---

8. Non-API background machinery (what APIs trigger)

Function	Called from	Does
`create_workflow_from_goal`	`workflows/launch`	`OrchestratorAgent.plan` → `create_workflow` → dispatch first `run_step`

Function	Called from	Does
(Celery)		
`run_step` (Celery, `agents_*` queues)	`create_workflow`, `advance_run`, `scan_due_steps`	Idempotency key → lead lock → `build_run_context` → `BaseAgent.run` (LangGraph) → `submit_proposal` → `advance_run`
`submit_proposal`	`run_step`	8-step policy gate → persist proposal → auto-exec or human gate
`execute_proposal` (Celery, executor queue)	`submit_proposal`, `decide`	`Executor.execute` → internal DB actions or connector (Meta/Google/WhatsApp/SES/content/search) → `mark_executed`
`scan_due_steps`, `expire_stale_proposals`, `reset_daily_budgets`	Celery beat	Periodic sweep, 48h expiry, daily spend reset
`whatsapp_webhook`	Meta webhook	Persist inbound message + conversation-agent workflow

---

Frontend → API map (`apps/web/src/api/services.ts`)

Page	API calls	Data consumed
Overview	`analyticsApi.*`	kpi counters, `{date,count}` series, funnel stages, status counts, audit events
AI Agent	`POST /workflows/launch {goal}`; `GET /workflows/`, `GET /workflows/{id}`	run `{id,goal,status,created`

Page	API calls	Data consumed
	(nested `steps`); `POST /workflows/{id}/{pause,resume,cancel}`; `GET /agents/`; `GET /proposals?status=pending`	`_at,steps:[{agent,state,run_at,completed_at,error_message}]}`
Approvals	`GET /proposals?status=`; `POST /proposals/{id}/decide {decision,note?,edited_payload?}`; `GET /analytics/proposal_stats`	proposal fields above; `by_status`, `avg_decision_seconds`
Workflows	`GET /workflows/`, `GET /workflows/{id}`	nested steps → React Flow nodes (color by `state`)
Leads	`GET /leads/?stage=`; `POST /leads/`; `GET /analytics/{funnel,lead_score_distribution}`	lead fields above; stage/score charts
CRM	`GET /conversations/`; `GET /conversations/{id}/messages/`; `GET /leads/`	inbox items, message bubbles, pipeline counts by `stage`
Advertising	`GET /analytics/ad_campaigns`, `spend_over_time`, `proposal_stats`; `POST /agents/run {agent:'paid_media',goal}`	campaigns or `{available:false,reason}`
Research	`POST /research/perform/`	`{topic,findings,sources,confidence,metadata}`
Knowledge	`POST /knowledge/store`, `POST /knowledge/search`	`{id}`, `{results:[{title,content,score}]}`
Module pages	`GET /agents/`, `POST /agents/run/ {agent,goal}`; `GET /proposals/`; `GET /analytics/trust`	agent registry, proposals filtered by `action_type` prefix, trust rows

Page	API calls	Data consumed
Integrations	`GET /analytics/integrations`	`{name,category,configured}`
Activity	`GET /analytics/activity?limit=200`	audit event rows
Calendar	`GET /steps/`	scheduled RunSteps sorted client-side by `run_at`
Assets	`GET /proposals/`	filtered `action_type` `content.` / `creative.`
Brand	`GET/POST /brands/`	brand records
Settings	—	static policy description

Data-shape notes for the frontend

- Paginated lists: always normalize `Array.isArray(data) ? data : data.results`
- `Proposal.payload` and `evidence` are free-form JSON
- Money is `spend_paise` (paise; divide by 100 for ₹)
- `risk_tier` int 0–3; `trust_level` int 1–3; `state` / `status` enums documented in serializers
- `ad_campaigns` may return `available:false` — never render it as empty data